新藥試驗存活分析報告

Kaplan-Meier 存活曲線、Log-rank 檢定與 Cox 風險比

資料來源：Survival test example.xlsx
報告日期：2026 年 10 月 29 日

目錄

摘要 3
1. 資料概述 3
2. 統計方法（摘要） 4
3. 分析結果 5
3.1 Kaplan-Meier 存活曲線 5
3.2 存活表（Life table） 5
3.3 中位存活時間與 RMST 6
3.4 組間比較：Log-rank 檢定 6
3.5 風險比（Cox 比例風險模型） 6
3.6 比例風險假設 7
4. 與 SPSS 結果比對 7
5. 結論 8
6. 交接檔（統計方法說明） 9
6.1 資料與變數定義 9
6.2 分析流程 9
6.3 方法細節與公式 9
6.4 軟體與檔案 10
6.5 交叉驗證建議 10
6.6 限制與注意事項 10
6.7 後續建議 11
附錄：完整分析程式（km_analysis.py） 11

摘要

組別定義：Group 1 = 安慰劑組，Group 2 = 新藥組（依研究者提供之資訊）。所有風險比以安慰劑組為參考組。

主要結果：新藥組中位存活時間 5.0 個月，短於安慰劑組的 6.5 個月。Cox 模型估計新藥相對於安慰劑的風險比 HR = 2.81（95% CI 0.69–11.46），點估計顯示新藥組的事件風險約為安慰劑組的 2.8 倍。

統計顯著性：Log-rank $\chi^2 = 2.304$，$p = 0.129$；Breslow 與 Tarone-Ware 檢定亦不顯著。在本樣本（n = 12、共 9 個事件）下，兩組存活差異未達統計顯著。

一句話結論：本資料不支持新藥能延長存活；點估計方向甚至不利於新藥，但因樣本過小、信賴區間極寬，無法確認新藥有害或無效。

與 SPSS 比對：存活表、中位數、平均存活時間及三項組間檢定皆與 SPSS 輸出完全一致（詳見第 4 章）。

1. 資料概述

本資料共 12 位受試者，安慰劑組（Group 1）與新藥組（Group 2）各 6 人。結果變項為追蹤時間（月）與事件狀態（1 = 發生事件，0 = 設限）。新藥組 6 人全數發生事件；安慰劑組有 3 人在追蹤結束時仍未發生事件（設限 50%）。

表 1　各組描述性統計

組別	人數	事件數	設限數	追蹤時間 中位數（月）	追蹤時間 範圍（月）
安慰劑（1）	6	3	3	6.25	4.5 – 7.2
新藥（2）	6	6	0	5.25	4.3 – 7.0

表 2　原始資料

ID	組別	追蹤時間（月）	狀態
A	1（安慰劑）	5.0	1 = 事件
B	1（安慰劑）	7.0	0 = 設限
C	1（安慰劑）	6.0	0 = 設限
D	1（安慰劑）	4.5	1 = 事件
E	1（安慰劑）	7.2	0 = 設限
F	1（安慰劑）	6.5	1 = 事件
c	2（新藥）	4.3	1 = 事件
a	2（新藥）	5.0	1 = 事件
d	2（新藥）	5.5	1 = 事件
e	2（新藥）	6.4	1 = 事件
b	2（新藥）	7.0	1 = 事件
f	2（新藥）	5.0	1 = 事件

2. 統計方法（摘要）

- **Kaplan-Meier 法：**估計各組存活函數 S(t)；標準誤採 Greenwood 公式，95% 信賴區間採 log(−log) 轉換。
- **中位存活時間：**S(t) ≤ 0.5 的最早時間點；其 95% CI 由信賴帶與 0.5 的交點求得（Brookmeyer-Crowley 型方法）。
- **組間比較：**主要檢定為 Log-rank（Mantel-Cox），雙尾，α = 0.05；另以 Breslow（Generalized Wilcoxon）與 Tarone-Ware 檢定作敏感度分析（與 SPSS 相同）。
- **風險比：**單變項 Cox 比例風險模型（安慰劑組為參考組），同分事件以 Efron 法處理；同時提供 Wald 與 profile likelihood 信賴區間。
- **模型假設：**以 Schoenfeld 殘差的 Grambsch-Therneau 檢定評估比例風險假設。
- **補充指標：**限制平均存活時間（RMST），截斷點 τ = 7.0 月（兩組最長追蹤時間之較小者）。

詳細公式、程式與注意事項請見第 6 章交接檔。

3. 分析結果

3.1 Kaplan-Meier 存活曲線

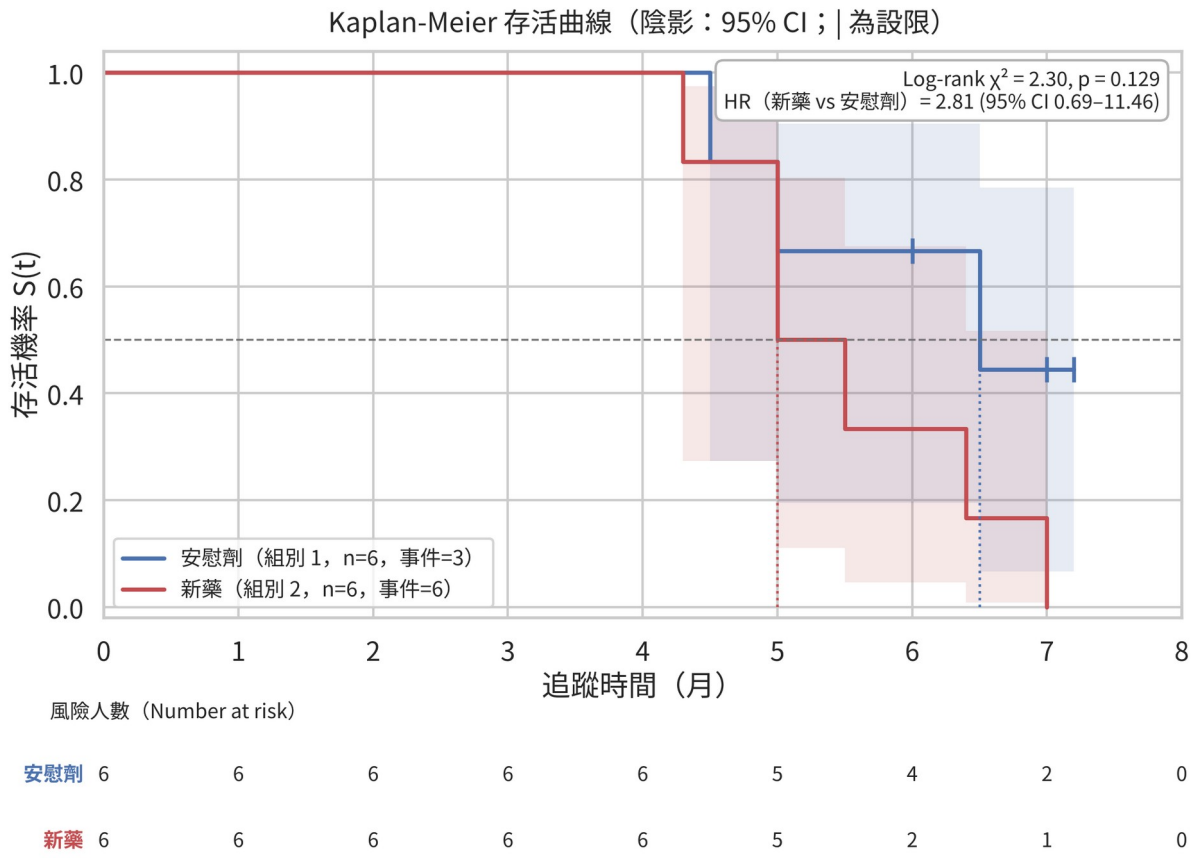


圖 1　兩組 Kaplan-Meier 存活曲線（陰影為 95% CI，直線標記為設限，虛線標示中位存活時間）

兩組在約 4.3 個月前皆無事件。之後新藥組曲線下降較快且較深，於 7.0 個月時存活率降至 0；安慰劑組在 6.5 個月後維持在 0.444。兩條曲線未交叉，但信賴區間大幅重疊，反映樣本數小、估計不確定性高。

3.2 存活表（Life table）

表 3　安慰劑組（Group 1）Kaplan-Meier 存活表

時間（月）	風險人數	事件	設限	S(t)	SE	95% CI
4.5	6	1	0	0.833	0.152	0.273 – 0.975
5.0	5	1	0	0.667	0.192	0.195 – 0.904
6.0	4	0	1	0.667	0.192	0.195 – 0.904
6.5	3	1	0	0.444	0.222	0.066 – 0.785
7.0	2	0	1	0.444	0.222	0.066 – 0.785
7.2	1	0	1	0.444	0.222	0.066 – 0.785

表 4　新藥組（Group 2）Kaplan-Meier 存活表

時間（月）	風險人數	事件	設限	S(t)	SE	95% CI
4.3	6	1	0	0.833	0.152	0.273 – 0.975
5.0	5	2	0	0.500	0.204	0.111 – 0.804
5.5	3	1	0	0.333	0.192	0.046 – 0.676
6.4	2	1	0	0.167	0.152	0.008 – 0.517
7.0	1	1	0	0.000	0.000	—

3.3 中位存活時間與 RMST

表 5　中位存活時間與限制平均存活時間

組別	中位存活時間（月）	95% CI（log-log，月）	RMST（τ = 7.0 月）
安慰劑	6.5	4.5 – NA	6.14
新藥	5.0	4.3 – 7.0	5.53

安慰劑組中位存活時間 6.5 個月，其 95% CI 上限無法估計（NA），因為上側信賴帶在追蹤期間內從未降至 0.5 以下；新藥組 5.0 個月。以 RMST 看，在前 7.0 個月內，安慰劑組平均無事件時間為 6.14 個月，新藥組為 5.53 個月，新藥組少約 0.61 個月。

3.4 組間比較：Log-rank 檢定

表 6　Log-rank 檢定之觀察與期望事件數

組別	觀察事件 O	期望事件 E	O / E
安慰劑	3	5.13	0.58
新藥	6	3.87	1.55

新藥組觀察到 6 個事件，多於虛無假設下的期望值 3.87；安慰劑組 3 個，少於期望值 5.13。Log-rank $\chi^2 = 2.304$，df = 1，p = 0.129。對早期事件給較大權重的 Breslow 與 Tarone-Ware 檢定 p 值更大（見表 8），因兩組早期（< 5 個月）差異很小，差距主要出現在後期。

3.5 風險比（Cox 比例風險模型）

表 7　風險比估計

比較	HR	95% CI（Wald）	95% CI（Profile likelihood）	p（Wald）
新藥 vs 安慰劑（參考）	2.81	0.69 – 11.46	0.73 – 13.53	0.150
安慰劑 vs 新藥（參考）	0.36	0.09 – 1.45	0.07 – 1.38	0.150
Peto 法 HR = exp[(O−E)/V]（新藥 vs 安慰劑）	2.94	—	—	0.129
O/E 比值法 HR =$(O_2/E_2)/(O_1/E_1)$	2.66	—	—	0.129

表 8　各項檢定彙整

檢定 / 指標	統計量	p 值
Log-rank (Mantel-Cox)	χ^2 = 2.304（df = 1）	0.129
Breslow (Generalized Wilcoxon)	χ^2 = 1.330（df = 1）	0.249
Tarone-Ware	χ^2 = 1.777（df = 1）	0.183
Cox 模型 Wald 檢定	z = 1.44	0.150
Cox 模型 概似比檢定	χ^2 = 2.23（df = 1）	0.135
Cox 模型 Score 檢定	χ^2 = 2.25（df = 1）	0.134
比例風險假設（Grambsch-Therneau）	χ^2 = 0.55（df = 1）	0.457

三種 Cox 檢定（Wald、概似比、Score）與 Log-rank 結果一致，p 值皆約 0.13–0.15。HR 點估計 2.81 顯示新藥組風險較高，但信賴區間跨越 1，且區間很寬（上限超過 11），表示資料與「無差異」以及「風險相差數倍」皆相容。

3.6 比例風險假設

Grambsch-Therneau 檢定 p = 0.457，未發現違反比例風險假設的證據；從圖 1 亦可見兩條曲線未交叉。惟本檢定在 9 個事件下檢定力很低，此結果僅能視為「未見明顯違反」。

4. 與 SPSS 結果比對

將本分析與研究者提供的 SPSS（KM 程序）輸出逐項比對如下：

表 9　SPSS 與本分析結果比對

項目	SPSS	本分析	一致性
存活表 S(t) 與 SE	安慰劑 .833/.667/.444；新藥 .833/.500/.333/.167/0	相同	✓ 完全一致
中位存活時間	安慰劑 6.500；新藥 5.000	6.500；5.000	✓ 完全一致
平均存活時間（截斷於各組最長時間）	安慰劑 6.228；新藥 5.533	6.228；5.533	✓ 完全一致
Log-rank (Mantel-Cox)	χ^2 2.304，p .129	χ^2 2.304，p 0.129	✓ 完全一致
Breslow (Generalized Wilcoxon)	χ^2 1.330，p .249	χ^2 1.330，p 0.249	✓ 完全一致
Tarone-Ware	χ^2 1.777，p .183	χ^2 1.777，p 0.183	✓ 完全一致
中位數 95% CI	安慰劑 3.56–9.44；新藥 4.04–5.96	4.5–NA；4.3–7.0	方法不同（見下）
風險比 HR	未執行（KM 程序不提供）	2.81（Efron）；2.74（Breslow）	本分析補充

差異說明一：中位數信賴區間。SPSS 以「中位數 ± 1.96 × SE」計算對稱區間，可能超出觀察範圍（如安慰劑上限 9.44 個月，超過最長追蹤 7.2 個月）。本分析採 Brookmeyer-Crowley 型方法，由存活曲線信賴帶與 0.5 的交點求得，區間不會超出資料範圍，且可正確反映上限無法估計（NA）的情形，小樣本時較為合理。兩種方法並無對錯，但報告時應註明所用方法。

差異說明二：平均存活時間 vs RMST。SPSS 的「平均值」為截斷於各組自己最長追蹤時間的 RMST（安慰劑 7.2、新藥 7.0 個月），兩組截斷點不同，嚴格來說不宜直接相減。本報告表 5 改以共同截斷點 τ = 7.0 月計算，安慰劑組為 6.139，與 SPSS 的 6.228 差異即來自 7.0–7.2 月這段（0.2 × 0.444 = 0.089）。

差異說明三：風險比。SPSS 的 KM 程序不輸出 HR，需另執行 COXREG。SPSS COXREG 以 Breslow 法處理同分，預期得到 HR ≈ 2.74（95% CI 0.67–11.20，p = 0.161）；本報告主要結果採 Efron 法（HR 2.81），與 R、Stata（efron 選項）、lifelines 預設一致。新藥組在 5.0 個月有 2 人同時發生事件，兩法因此略有差異，結論不變。

5. 結論

- 新藥組的存活表現在數值上「劣於」安慰劑組：中位存活時間 5.0 vs 6.5 個月，HR = 2.81（新藥 vs 安慰劑），新藥組 6 人全數發生事件，安慰劑組僅 3 人。
- 此差異未達統計顯著（Log-rank p = 0.129；HR 95% CI 0.69–11.46 包含 1），Breslow 與 Tarone-Ware 檢定結論相同。
- 結論：本資料沒有證據顯示新藥能延長存活；點估計方向不利於新藥，但樣本極小（每組 6 人、共 9 事件），既無法證明新藥有害，也無法排除具臨床意義的差異。建議檢視新藥組的安全性資料，並以較大樣本再評估。

6. 交接檔（統計方法說明）

本章提供接手人員重現與延伸本分析所需的全部資訊。

6.1 資料與變數定義

Excel 欄位	程式變數	型態	定義
ID	ID	文字	受試者代碼（大小寫視為不同人，如 A 與 a）
Group	group	整數	1 = 安慰劑（Cox 參考組）；2 = 新藥
Follow-up Months	time	數值	自納入至事件或最後追蹤的時間（月）
Status 1=event 0=censor	event	整數	1 = 發生事件；0 = 右設限

待確認事項：(1)「事件」的臨床定義（死亡、疾病惡化等）；(2) 時間零點定義（隨機分派日或首次給藥日）；(3) 是否為隨機分派。這些會影響結論的解讀，但不影響數值結果。

6.2 分析流程

1. 讀取 Excel，統一欄位名稱，確認無遺漏值。
2. 各組分別計算 Kaplan-Meier 存活表（風險人數、事件數、S(t)、Greenwood SE、log(−log) 95% CI）。
3. 由存活表求中位存活時間及其 CI；計算 τ = 7.0 月之 RMST。
4. Log-rank 檢定比較兩組存活分布；Breslow、Tarone-Ware 為敏感度分析。
5. 建立單變項 Cox 模型（x = 1 表新藥組，安慰劑為參考），以 Newton-Raphson 求最大部分概似估計，計算 HR、Wald / profile likelihood CI 及三種檢定。
6. 以 Schoenfeld 殘差檢定比例風險假設。
7. 以 seaborn / matplotlib 繪製 KM 曲線並加上風險人數表；結果輸出 results.json 與 km_curve.png。

6.3 方法細節與公式

Kaplan-Meier 估計式：$S(t) = \Pi_{t_i \le t} (1 - d_i / n_i)$，其中 d_i 為時間 t_i 的事件數，n_i 為 t_i 前一刻的風險人數。同一時間的設限視為發生在事件之後（仍計入該時點的風險集合）。

Greenwood 變異數：$\mathrm{Var}[S(t)] = S(t)^2 \cdot \Sigma\, d_i / [n_i (n_i - d_i)]$。

log(−log) 信賴區間：$SE^* = \sqrt{(\Sigma\, d_i/[n_i(n_i-d_i)])} / |\log S(t)|$，CI = S(t)^exp(±1.96·SE*)。此轉換保證區間位於 0 與 1 之間，為 lifelines 與 R survival 套件（conf.type = "log-log"）可選之方法。R 的預設為 "log"，因此與 R 預設輸出比對時 CI 會略有不同。

中位存活 CI：下限 = CI 下限首次 ⩽ 0.5 的時間；上限 = CI 上限首次 ⩽ 0.5 的時間；若從未 ⩽ 0.5 則為 NA。

Log-rank 檢定：在每個事件時間點計算安慰劑組（Group 1）的期望事件數 $E = d \cdot n_1/n$ 與超幾何變異數 $V = d \cdot (n_1/n)(1-n_1/n)(n-d)/(n-1)$，統計量 $\chi^2 = (O_1 - E_1)^2 / \Sigma V$，自由度 1。

Cox 模型： $h(t \mid x) = h_0(t)\cdot\exp(\beta x)$。同分事件採 Efron 近似（與 R coxph 及 lifelines 預設一致）。HR = exp(β)，Wald CI = exp(β ± 1.96·SE)。Profile likelihood CI 為使 $2[\ell(\hat\beta) - \ell(\beta)] = 3.84$ 的 β 值，小樣本時較 Wald CI 可靠。程式內以數值微分驗證解析 score 函數在 $\hat\beta$ 處為 0。

比例風險檢定： 以 Breslow 型 Schoenfeld 殘差 r_k 與事件時間（未轉換）做 score 檢定：$\chi^2 = [\Sigma(g_k - \bar g)\, r_k]^2 / [\Sigma\, g_k'^2\, V_k - (\Sigma\, g_k'\, V_k)^2/\Sigma\, V_k]$。R cox.zph 預設使用 KM 轉換的時間，數值可能略有差異。

Breslow / Tarone-Ware 檢定： 與 log-rank 相同架構，但每個事件時間的 (O − E) 與變異數分別乘以權重 n_i 與 $\sqrt{n_i}$，因此對早期事件（風險人數多時）給較大權重。

RMST： τ 以內 KM 曲線下的面積，τ = 兩組最大追蹤時間之較小者（7.0 月）。

6.4 軟體與檔案

項目	說明
語言 / 套件	Python 3；numpy、pandas、scipy、matplotlib、seaborn 0.13、openpyxl
為何未用 lifelines	執行環境無法安裝 lifelines，因此所有估計式於 km_analysis.py 內自行實作，公式見 6.3
分析程式	km_analysis.py（隨報告附上）
執行方式	python km_analysis.py "Survival test example.xlsx" output
輸出檔	output/km_curve.png（圖 1）、output/results.json（全部數值）
Word 產生	build_docx.js（Node.js docx 套件，讀取 results.json 產生本報告）

6.5 交叉驗證建議

若需以標準套件複核，以下 R 程式應得到相同的 KM 估計、log-rank χ^2 與 Cox HR：

```
library(survival); library(readxl)
d <- read_excel("Survival test example.xlsx"); names(d) <-
c("ID","group","time","event")
fit <- survfit(Surv(time, event) ~ group, data = d, conf.type = "log-log")
summary(fit); print(fit)                          # 存活表與中位數
survdiff(Surv(time, event) ~ group, data = d)  # Log-rank
m <- coxph(Surv(time, event) ~ factor(group), data = d, ties = "efron")
summary(m); cox.zph(m)                             # HR 與比例風險假設
```

Python 使用者可改用 lifelines：KaplanMeierFitter、logrank_test、CoxPHFitter（預設 Efron 法）。

6.6 限制與注意事項

- 樣本數極小（n = 12，9 事件），所有估計的信賴區間都很寬，檢定力不足以偵測中等大小的效果。
- 新藥組無任何設限，其最後時點 S(t) = 0 為「最後一位受試者發生事件」所致，不代表真實存活率為 0。
- 安慰劑組在 7.2 個月後無資料，曲線尾端估計僅基於 1–2 人。
- Cox 模型僅含組別一個變項，未調整任何干擾因子；若為非隨機分組，結果可能有偏差。

- 設限假設為非資訊性（censoring 與預後無關），需由研究設計確認。
- Wald 檢定在小樣本下可能不穩定，建議優先參考概似比檢定與 profile likelihood CI。

6.7 後續建議

- 確認事件定義與是否隨機分派後，於正式報告中補充說明。
- 若需在 SPSS 中取得 HR：分析 → 存活 → Cox 迴歸，Group 設為類別變數並以第一類（安慰劑）為參考。
- 若有更多受試者或共變數（年齡、性別、疾病分期），可擴充為多變項 Cox 模型。
- 規劃正式試驗時，可依本資料觀察到的 HR 進行樣本數估算（例如欲偵測 HR = 0.6、α = 0.05、檢定力 80%，約需 120 個事件，依 Schoenfeld 公式 $4 \cdot (1.96+0.84)^2 / (\ln 0.6)^2 \approx 120$）。

附錄：完整分析程式（km_analysis.py）

```
"""
Kaplan-Meier / Log-rank / Cox PH analysis for "Survival test example.xlsx"
Dependencies: numpy, pandas, scipy, matplotlib, seaborn, openpyxl
(lifelines was unavailable, so all estimators are implemented explicitly below.)

Usage: python km_analysis.py "Survival test example.xlsx" outdir
"""
import sys, json, os
import numpy as np
import pandas as pd
from scipy import stats, optimize
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
import seaborn as sns

IN = sys.argv[1] if len(sys.argv) > 1 else "Survival test example.xlsx"
OUT = sys.argv[2] if len(sys.argv) > 2 else "output"
os.makedirs(OUT, exist_ok=True)
Z = stats.norm.ppf(0.975)

# ---------------------------------------------------------------- data
raw = pd.read_excel(IN)
df = raw.rename(columns={raw.columns[0]: "ID", raw.columns[1]: "group",
                         raw.columns[2]: "time", raw.columns[3]: "event"})
df["group"] = df["group"].astype(int)
df["event"] = df["event"].astype(int)
groups = sorted(df["group"].unique())
LABEL = {1: "安慰劑", 2: "新藥"}   # 1 = placebo, 2 = new drug (confirmed by user)


# ---------------------------------------------------------------- Kaplan-Meier
def kaplan_meier(t, e):
    """KM estimate with Greenwood variance and log(-log) 95% CI."""
    t, e = np.asarray(t, float), np.asarray(e, int)
    rows, S, gw = [], 1.0, 0.0
```

```
    for u in np.unique(t):
        n = (t >= u).sum()
        d = ((t == u) & (e == 1)).sum()
        c = ((t == u) & (e == 0)).sum()
        if d > 0:
            S *= 1 - d / n
            gw += d / (n * (n - d)) if n > d else np.inf
        if 0 < S < 1 and np.isfinite(gw):
            se_ll = np.sqrt(gw) / abs(np.log(S))          # SE of log(-log S)
            lo, hi = S ** np.exp(Z * se_ll), S ** np.exp(-Z * se_ll)
        elif S == 1:
            lo = hi = 1.0
        else:
            lo = hi = np.nan if S > 0 else 0.0
        se = S * np.sqrt(gw) if np.isfinite(gw) else (0.0 if S == 0 else np.nan)
        rows.append(dict(time=u, n_risk=n, n_event=d, n_censor=c,
                         surv=S, se=se, lower=lo, upper=hi))
    return pd.DataFrame(rows)


def km_quantile(tab, col, p=0.5):
    """Smallest time at which curve `col` is <= p (NaN if never reached)."""
    hit = tab.loc[tab[col] <= p, "time"]
    return float(hit.iloc[0]) if len(hit) else np.nan


km = {g: kaplan_meier(df.loc[df.group == g, "time"], df.loc[df.group == g,
"event"])
      for g in groups}

medians = {}
for g in groups:
    tab = km[g].dropna(subset=["surv"])
    # Brookmeyer-Crowley-type CI: times where CI band crosses 0.5
    medians[g] = dict(median=km_quantile(tab, "surv"),
                      lower=km_quantile(tab, "lower"),
                      upper=km_quantile(tab, "upper"))

# ---------------------------------------------------------------- Log-rank
def logrank(df):
    t, e, g = df.time.values, df.event.values, df.group.values
    g1 = groups[0]
    O = E = V = 0.0
    for u in np.unique(t[e == 1]):
        at = t >= u
        n, n1 = at.sum(), (at & (g == g1)).sum()
        d = ((t == u) & (e == 1)).sum()
        d1 = ((t == u) & (e == 1) & (g == g1)).sum()
        O += d1
        E += d * n1 / n
        if n > 1:
            V += d * (n1 / n) * (1 - n1 / n) * (n - d) / (n - 1)
    chi2 = (O - E) ** 2 / V
    tot = {gg: int(((g == gg) & (e == 1)).sum()) for gg in groups}
```

```
    E2 = sum(tot.values()) - E
    return dict(O1=O, E1=E, O2=tot[groups[1]], E2=E2, var=V, chi2=chi2,
                p=stats.chi2.sf(chi2, 1),
                hr_mh=np.exp((tot[groups[1]] - E2) / V),  # Peto/MH approx (grp2 vs
grp1)
                )

lr = logrank(df)

def weighted_test(df, wfun):
    t, e, g = df.time.values, df.event.values, df.group.values
    U = V = 0.0
    for u in np.unique(t[e == 1]):
        at = t >= u
        n, n1 = at.sum(), (at & (g == groups[0])).sum()
        d = ((t == u) & (e == 1)).sum()
        d1 = ((t == u) & (e == 1) & (g == groups[0])).sum()
        w = wfun(n)
        U += w * (d1 - d * n1 / n)
        if n > 1:
            V += w * w * d * (n1 / n) * (1 - n1 / n) * (n - d) / (n - 1)
    return dict(chi2=U * U / V, p=stats.chi2.sf(U * U / V, 1))
tests = {"Log-rank (Mantel-Cox)": dict(chi2=lr["chi2"], p=lr["p"]),
         "Breslow (Generalized Wilcoxon)": weighted_test(df, lambda n: n),
         "Tarone-Ware": weighted_test(df, lambda n: np.sqrt(n))}

# ------------------------------------------------------------------ Cox PH (Efron
ties)
x = (df.group.values == groups[1]).astype(float)       # 1 = group 2, 0 = group 1
(reference)
tt, ee = df.time.values, df.event.values


def cox_efron(beta, breslow=False):
    ll = U = I = 0.0
    for u in np.unique(tt[ee == 1]):
        risk = tt >= u
        dmask = (tt == u) & (ee == 1)
        d = dmask.sum()
        r = np.exp(beta * x)
        s0, s1, s2 = r[risk].sum(), (r * x)[risk].sum(), (r * x * x)[risk].sum()
        t0, t1, t2 = r[dmask].sum(), (r * x)[dmask].sum(), (r * x * x)[dmask].sum()
        ll += beta * x[dmask].sum()
        U += x[dmask].sum()
        for l in range(d):
            f = 0.0 if breslow else l / d
            a0, a1, a2 = s0 - f * t0, s1 - f * t1, s2 - f * t2
            ll -= np.log(a0)
            U -= a1 / a0
            I += a2 / a0 - (a1 / a0) ** 2
    return ll, U, I


b = 0.0
```

```
for _ in range(50):                                        # Newton-Raphson
    _, U, I = cox_efron(b)
    step = U / I
    b += step
    if abs(step) < 1e-10:
        break
ll1, _, I1 = cox_efron(b)
ll0, U0, I0 = cox_efron(0.0)
se_b = 1 / np.sqrt(I1)
cox = dict(beta=b, se=se_b, HR=np.exp(b),
           HR_lo=np.exp(b - Z * se_b), HR_hi=np.exp(b + Z * se_b),
           z=b / se_b, p_wald=2 * stats.norm.sf(abs(b / se_b)),
           LR=2 * (ll1 - ll0), p_LR=stats.chi2.sf(2 * (ll1 - ll0), 1),
           score=U0 ** 2 / I0, p_score=stats.chi2.sf(U0 ** 2 / I0, 1),
           loglik=ll1, loglik0=ll0)
cox["HR_inv"] = 1 / cox["HR"]
cox["HR_inv_lo"], cox["HR_inv_hi"] = 1 / cox["HR_hi"], 1 / cox["HR_lo"]

# profile-likelihood CI (more reliable than Wald with n = 12)
def pl_root(target):
    return lambda bb: 2 * (ll1 - cox_efron(bb)[0]) - stats.chi2.ppf(0.95, 1)
cox["PL_lo"] = np.exp(optimize.brentq(pl_root(None), b - 10, b))
cox["PL_hi"] = np.exp(optimize.brentq(pl_root(None), b, b + 10))

# Breslow ties (SPSS COXREG default) for comparison
bb = 0.0
for _ in range(50):
    _, U, I = cox_efron(bb, True); st = U / I; bb += st
    if abs(st) < 1e-10: break
seb = 1 / np.sqrt(cox_efron(bb, True)[2])
cox["breslow"] = dict(beta=bb, se=seb, HR=np.exp(bb), lo=np.exp(bb - Z * seb),
hi=np.exp(bb + Z * seb),
                      p=2 * stats.norm.sf(abs(bb / seb)))

# numerical check of analytic score/information
eps = 1e-5
num_U = (cox_efron(b + eps)[0] - cox_efron(b - eps)[0]) / (2 * eps)
assert abs(num_U) < 1e-5, num_U

# ------------------------------------------------------------------ PH assumption
(Schoenfeld / Grambsch-Therneau, Breslow approx)
res, times_ev, Vk = [], [], []
r = np.exp(b * x)
for i in np.where(ee == 1)[0]:
    risk = tt >= tt[i]
    xbar = (r * x)[risk].sum() / r[risk].sum()
    x2 = (r * x * x)[risk].sum() / r[risk].sum()
    res.append(x[i] - xbar); times_ev.append(tt[i]); Vk.append(x2 - xbar ** 2)
res, times_ev, Vk = map(np.array, (res, times_ev, Vk))
g_ = times_ev - times_ev.mean()
num = (g_ * res).sum() ** 2
den = (g_ ** 2 * Vk).sum() - (g_ * Vk).sum() ** 2 / Vk.sum()
ph = dict(chi2=num / den, p=stats.chi2.sf(num / den, 1))
```

```
# ---------------------------------------------------------------- RMST (tau = 7.0)
TAU = float(min(df.groupby("group").time.max()))
def rmst(tab, tau):
    ts = np.r_[0, tab.time.values[tab.time.values < tau], tau]
    ss = np.r_[1, tab.surv.values[tab.time.values < tau]]
    return float((np.diff(ts) * ss).sum())
rm = {g: rmst(km[g], TAU) for g in groups}
rm_spss = {g: rmst(km[g], float(km[g].time.max())) for g in groups}

# ---------------------------------------------------------------- descriptives
desc = df.groupby("group").agg(n=("ID", "size"), events=("event", "sum"),
                               fu_mean=("time", "mean"), fu_median=("time",
"median"),
                               fu_min=("time", "min"), fu_max=("time",
"max")).reset_index()
desc["censored"] = desc["n"] - desc["events"]

# ---------------------------------------------------------------- figure
fp = [f for f in font_manager.findSystemFonts() if "NotoSansCJK-Regular" in f]
if fp:
    font_manager.fontManager.addfont(fp[0])
    cjk = font_manager.FontProperties(fname=fp[0]).get_name()
else:
    cjk = "sans-serif"
sns.set_theme(style="whitegrid", context="talk",
              rc={"font.family": cjk, "axes.unicode_minus": False})
pal = sns.color_palette("deep")
colors = {groups[0]: pal[0], groups[1]: pal[3]}

fig = plt.figure(figsize=(11, 8.2), dpi=200)
ax = fig.add_axes([0.11, 0.33, 0.85, 0.6])
for g in groups:
    tab = km[g]
    ts = np.r_[0, tab.time.values]
    ss = np.r_[1, tab.surv.values]
    lo = np.r_[1, tab.lower.fillna(0).values]
    hi = np.r_[1, tab.upper.fillna(0).values]
    lo = np.where(np.isnan(lo), 0, lo)
    lab = f"{LABEL[g]}（組別 {g}，n={int((df.group == g).sum())}，事件
={int(df[(df.group == g)].event.sum())}）"
    ax.step(ts, ss, where="post", color=colors[g], lw=2.6, label=lab)
    ax.fill_between(ts, lo, hi, step="post", color=colors[g], alpha=0.13, lw=0)
    cen = tab[tab.n_censor > 0]
    ax.plot(cen.time, cen.surv, "|", color=colors[g], ms=16, mew=2.4)
ax.axhline(0.5, ls="--", lw=1.2, color="0.45")
for g in groups:
    m = medians[g]["median"]
    if np.isfinite(m):
        ax.plot([m, m], [0, 0.5], ls=":", lw=1.6, color=colors[g])
ax.set_xlim(0, 8); ax.set_ylim(-0.02, 1.04)
ax.set_xticks(range(0, 9))
ax.set_xlabel("追蹤時間（月）"); ax.set_ylabel("存活機率 S(t)")
ax.set_title("Kaplan-Meier 存活曲線（陰影: 95% CI; | 為設限）", fontsize=17, pad=12)
ax.legend(loc="lower left", fontsize=13, frameon=True)
```

```
txt = (f"Log-rank χ² = {lr['chi2']:.2f}, p = {lr['p']:.3f}\n"
       f"HR（新藥 vs 安慰劑）= {cox['HR']:.2f} "
       f"(95% CI {cox['HR_lo']:.2f}–{cox['HR_hi']:.2f})")
ax.text(0.98, 0.97, txt, transform=ax.transAxes, ha="right", va="top",
fontsize=12.5,
        bbox=dict(boxstyle="round,pad=0.4", fc="white", ec="0.7"))

# number-at-risk table
axr = fig.add_axes([0.11, 0.06, 0.85, 0.14]); axr.set_xlim(0, 8); axr.set_ylim(-
0.5, 1.5)
axr.axis("off")
axr.text(-0.4, 1.95, "風險人數（Number at risk）", fontsize=12.5, ha="left",
va="center")
for row, g in enumerate(groups):
    yy = 1 - row
    axr.text(-0.15, yy, f"{LABEL[g]}", ha="right", va="center", fontsize=12.5,
color=colors[g],
             fontweight="bold")
    for tick in range(0, 9):
        n = int(((df.group == g) & (df.time >= tick)).sum())
        axr.text(tick, yy, str(n), ha="center", va="center", fontsize=12.5)
fig.savefig(f"{OUT}/km_curve.png", bbox_inches="tight")
plt.close(fig)

# ------------------------------------------------------------------ save results
out = dict(desc=desc.to_dict("records"),
           km={str(g): km[g].to_dict("records") for g in groups},
           medians={str(g): medians[g] for g in groups},
           logrank=lr, tests=tests, rm_spss={str(g): rm_spss[g] for g in groups},
cox=cox, ph=ph, rmst={str(g): rm[g] for g in groups}, tau=TAU,
           data=df.to_dict("records"))
with open(f"{OUT}/results.json", "w") as f:
    json.dump(out, f, ensure_ascii=False, indent=1, default=float)
print(json.dumps({k: out[k] for k in ["medians", "tests", "rm_spss", "cox", "ph",
"rmst", "tau"]},
                 indent=1, default=float, ensure_ascii=False))
for g in groups:
    print(g); print(km[g].round(4).to_string(index=False))
```